A Twelve-Stage Verification Pipeline for Computational Physics Research

John Roehm

Draft — April 2026

Abstract

We describe a twelve-stage pipeline for conducting computational physics research with continuous machine verification. The pipeline integrates Python numerics, Lean 4 formal proofs, and the Aristotle automated theorem prover into a unified workflow where every formula has a formal theorem, every computed quantity has verified bounds, and every paper claim traces to computation. Applied to a research program spanning dissipative effective field theory, analog Hawking radiation, emergent gravity, quantum groups, and topological quantum field theory, the pipeline has produced 26355 machine-checked theorems across 2036 Lean modules with 0 remaining gaps (0.0%), while maintaining 16 automated validation checks and full parameter provenance. We present the architecture, its ten invariants, the automated theorem prover integration that routinely compresses months of manual proof work to hours, and a case study where the pipeline caught a real convention bug in pentagon equation verification. The methodology is domain-agnostic and immediately applicable to any research program combining numerical computation with formal verification.

1 Introduction

Computational physics research faces a growing reproducibility crisis. Numerical results are published without verified bounds, paper claims diverge from underlying computation over time, and there is no machine-checkable connection between theoretical statements and their computational implementations. The standard workflow—write code, produce figures, write paper, submit—provides no structural guarantee that the published claims match the actual computation.

We address this with a twelve-stage pipeline that makes verification *continuous* rather than post-hoc. Each stage has a gate that must pass before proceeding to the next, and ten invariants must hold at all times. The pipeline integrates three verification layers: Python numerics for fast iteration, Lean 4 formal proofs for mathematical certainty, and the Aristotle automated theorem prover for filling proof gaps.

We have applied this pipeline to a research program spanning 42 papers, producing:

- 26355 machine-checked theorems across 2036 Lean modules (0.0% gap rate)

- 322 theorems proved by the Aristotle automated prover across 44 submitted runs
- 16 automated cross-layer validation checks
- Full parameter provenance for every experimental value
- Zero heartbeat overrides (no performance workarounds in the proof system)

The pipeline is domain-agnostic: it requires only that the research combines numerical computation with formal claims. We discuss applicability to pharmaceutical research (FDA SaMD algorithmic vigilance), financial modeling, and climate science.

2 Pipeline Architecture

The twelve stages execute in strict order, each with a gate:

Stage	Name	Gate
1	Constants & Parameters	imports succeed
2	Formulas	every function has Lean ref
3a	Lean — Interactive Mcp Loop	proof closes OR decomposed to sector stubs
3b	Lean — Sorry Registration	lake build compiles, SORRY_GAPS accurate
5	Lean Build Verification	zero sorry, counts match
6	Python Tests	all tests pass
7	Cross-Layer Validation	validate.py all checks pass
8	Visualizations	all PNGs generated
9	Figure Review	LLM review all PASS
10	Paper Draft	paper_provenance check passes
11	Notebooks	notebook_exec + viz_consistency pass
12	Document Sync	full validate.py passes, counts consistent
13	Adversarial Review	fresh-context Opus sweep shows zero BLOCKERs

Table 1: Pipeline stages with their gates (autogenerated from `WAVE_EXECUTION_PIPELINE.md`).

The canonical source of truth principle is fundamental: physical constants live in exactly one file (`constants.py`), formulas in exactly one file (`formulas.py`), and figure functions in exactly one file (`visualizations.py`). All other code imports from these canonical sources. This prevents the drift that plagues research codebases where the same formula appears in multiple files with subtle variations.

3 Pipeline Invariants

Ten invariants must hold at *all times*, not just at stage completion:

1. **Canonical formulas.** `formulas.py` is the only place physics formulas live. All other code imports from it.

2. **Canonical constants.** `constants.py` is the only place experimental parameters and the theorem registry live.

3. **Canonical visualizations.** `visualizations.py` is the only place figure functions live.

4. **Every formula has a Lean theorem.** Every Lean theorem has a proof (zero sorry target).

5. **Every computed quantity has bounds.** Physical bounds are tested and enforced by validation.

6. **Every paper claim traces to computation.** Numerical claims match formula output within 0.5%.

7. **Narrative derives from data.** Feasibility claims need computed support; no hallucinated optimism.

8. **Every parameter has verified provenance.** Each value traces to a published source.

9. **Placeholder theorems are non-load-bearing.** Theorems proved as `True := trivial` are documentation markers only.

10. **No heartbeat overrides.** Expensive typeclass synthesis is resolved via architectural fixes, not workarounds.

Invariant 9 addresses a subtle inflation problem: 26 of our 26355 theorems are `True := trivial` placeholders that serve as documentation markers. These are tracked in a registry (`PLACEHOLDER_THEOREMS`) and excluded from substantive theorem counts; the substantive count is 26329. The automated counting system (`update_counts.py`) identifies them via the Lean environment's type metadata.

4 Three-Layer Verification

The three verification layers interact in a discovery–formalization–automation cycle:

Layer 1: Python numerics. Fast iteration for physical intuition. A formula is implemented in `formulas.py` with a docstring referencing the planned Lean theorem name. Python tests verify correctness, edge cases, and physical bounds.

Layer 2: Lean 4 formal proofs. Mathematical certainty. The formula is stated as a Lean theorem, initially with `sorry` placeholders. The theorem statement is the strongest form possible—no weakening for proof convenience. Manual proofs are written for theorems within reach.

Layer 3: Aristotle automated prover. Fills sorry gaps. The entire Lean project is submitted with strategy hints ("PROVIDED SOLUTION" in docstrings). Aristotle uses tactic synthesis, term search, and machine learning to find proofs. Results are manually reviewed before integration—no blind acceptance.

Example: Tetrad gap equation. The critical coupling $G_c = 8\pi^2/(N_f\Lambda^2)$ was derived in Python (Layer 1), stated as a Lean theorem with sorry (Layer 2), and proved by Aristotle in run `79e07d55` (Layer 3). The three-layer cycle completed in one session.

5 Automated Theorem Prover Integration

Aristotle is a commercial automated theorem prover for Lean 4. Our integration follows a priority batching protocol:

Priority batching. Sorry gaps are organized into batches by priority (P1 = highest) and submitted sequentially. Each submission sends the entire Lean project, so parallel submissions waste budget. The current batch plan has 33 sorry across 4 batches.

PROVIDED SOLUTION hints. Each sorry theorem includes a docstring describing the expected proof strategy. This guides Aristotle's search and dramatically improves success rates. For example, the Serre coproduct hint describes the 4-phase tactic strategy that worked for the rank-1 case.

Quality verification. Every Aristotle result is manually reviewed before integration:

- Does the proof exercise the hypotheses? (Not vacuous)
- Does `lake build` pass after integration?
- Are there any heartbeat overrides? (Invariant 10 violation)

Results. 322 theorems proved automatically across 44 runs. The most impactful single run (`78dcc5f4`) proved 25 theorems spanning S-matrix unitarity, restricted quantum groups, ribbon categories, spin bordism, E8 lattice verification, and verified statistics in a single session. The compression ratio—months of estimated manual proof work compressed to hours of automated proving—is the pipeline's most dramatic efficiency gain.

Failure mode. Aristotle occasionally returns OUT_OF_BUDGET, indicating that the proof search exceeded the allocated computation. The resolution is to split the batch, improve hints, or attempt manual proof. One notable failure led to the discovery of a convention bug (Section 8).

6 Validation Infrastructure

The `validate.py` script runs 16 automated checks:

In addition, two LLM-powered review agents provide semantic validation:

- **Claims-reviewer**: Reads each paper's `.tex` file and cross-references numerical values against computation, Lean theorem status, and parameter provenance.
- **Figure-reviewer**: Inspects generated figures for rendering issues, label accuracy, and consistency with the data.

7 Automated Counts System

A recurring source of inefficiency was manual synchronization of theorem counts across 10+ documentation files. We resolved this with an automated counting system:

1. `ExtractDeps.lean`: A meta-programming script that extracts the full declaration taxonomy from the Lean environment, producing `lean_deps.json`.
2. `update_counts.py`: Reads `lean_deps.json` plus Python directory scans, producing `counts.json` (single source of truth) and `counts.tex` (LaTeX macros).
3. Papers use `\input{counts.tex}` and macros like `\totaltheorems` for auto-updated counts.

The system distinguishes substantive theorems from placeholders by checking the Lean type: theorems with type `True` are classified as placeholders. This eliminated a $\sim$4% count inflation from documentation-marker theorems.

8 Case Study: The Pentagon Convention Bug

The pipeline's most dramatic contribution was catching a convention bug in the Ising MTC pentagon equation.

Phase 1: False proof. The Aristotle automated prover solved the pentagon equation using case analysis (`fin_cases` + `ring`), producing a proof that compiled and type-checked. The proof appeared valid.

Phase 2: Discovery. When we rewrote the F-symbols using a different convention (switching from a phase-ambiguous source to Kitaev's canonical Appendix E convention), the pentagon equation failed. Investigation revealed that the original Aristotle proof was *vacuously true*: the F-symbol function returned incorrect values for $\sim$55 of 243 input tuples, making the pentagon equation hold trivially (both sides evaluated to zero for inadmissible inputs).

Phase 3: Resolution. Deep research identified the correct convention (Kitaev 2006, Appendix E), including the two crucial -1 entries ($F_{\sigma}^{\psi\sigma\psi}[\sigma,\sigma] = -1$ and $F_{\psi}^{\sigma\psi\sigma}[\sigma,\sigma] = -1$). The corrected F-symbols were verified by `native_decide` over $\mathbb{Q}(\sqrt{2})$ in $\sim$2 seconds. All 243 pentagon instances now check against the actual algebraic identities, not vacuously.

Lesson. A formally verified proof can still be wrong if the *definitions* it operates on are wrong. The pipeline's multi-layer approach—where the same mathematical content is expressed in both Python (numerical) and Lean (formal)—creates cross-checks that catch such errors. The convention bug would not have been detected by formal verification alone; it required the interaction between computational verification (`native_decide`) and the formalization infrastructure.

9 Scaling and Applicability

Current scale. The pipeline manages 26355 theorems, 2036 Lean modules, 42 papers, 91 notebooks, 170 figures, and 137 Python modules. Growth rate is approximately 200

theorems per intensive session. The automated counts system and validation infrastructure scale linearly.

Bottlenecks. Three factors limit throughput: (1) Aristotle budget constraints for proof search, (2) Lean 4 typeclass synthesis for complex categorical hierarchies (resolved by architectural caching, not heartbeat overrides), and (3) number field consolidation for algebraic verification at higher degree extensions.

Applicability. The pipeline requires only two ingredients: (1) numerical computation producing quantitative claims, and (2) formal statements of the mathematical content. Domains where this applies include:

- **Pharmaceutical research**: FDA Software as a Medical Device (SaMD) regulation requires algorithmic transparency. The pipeline's provenance tracking and cross-layer validation directly address algorithmic vigilance requirements.
- **Financial modeling**: Verified risk calculations where model drift from documented methodology is a regulatory concern.
- **Climate science**: Verified model predictions where the connection between code and published projections must be auditable.

10 Related Work

Physics formalizations. PhysLean (formerly HepLean) [1] formalizes the quantum harmonic oscillator and Higgs potential in Lean 4 but does not include a verification pipeline or cross-layer validation. The Lean-QuantumInfo project (Perimeter Institute) formalizes quantum information theory.

Verified computation. CompCert [2] verifies a C compiler; seL4 [3] verifies an operating system kernel. These are single-artifact verification projects, whereas our pipeline verifies an ongoing research program with continuously evolving content.

Reproducibility tools. Jupyter notebooks, papermill, and DVC provide computational reproducibility without formal verification. Our pipeline adds the formal layer: not just "the code runs and produces the same output" but "the output satisfies machine-checked mathematical properties."

Automated theorem proving. Lean Copilot [4] and AlphaProof [5] represent the frontier of AI-assisted theorem proving. Aristotle occupies a complementary niche: rather than proving competition problems, it fills sorry gaps in applied formalization projects where strategy hints significantly constrain the search space.

11 Conclusion

The twelve-stage pipeline transforms computational physics methodology from "write code, produce figures, write paper" to a continuous verification process where every stage has a machine-checkable gate. The ten invariants prevent the drift between computation and publication that plagues traditional research workflows. The three-layer verification architecture

(Python + Lean + Aristotle) provides both fast iteration and mathematical certainty, with the automated prover compressing proof work by orders of magnitude.

The pipeline caught a real convention bug in pentagon equation verification that would have propagated undetected through traditional methods. This single instance justifies the verification overhead: the cost of finding and fixing the bug *after publication* would have been far greater than the cost of continuous verification.

The methodology is open-source, domain-agnostic, and immediately applicable. We encourage adoption in any research domain where the connection between computation and published claims must be trustworthy.

References

[1] J. Tooby-Smith, "PhysLean: Lean 4 for physics," `https://github.com/HEPLean/PhysLean` (2024).

[2] X. Leroy, "Formal verification of a realistic compiler," *Comm. ACM* **52**(7), 107–115 (2009).

[3] G. Klein et al., "seL4: Formal verification of an OS kernel," *Proc. ACM SOSP* (2009).

[4] P. Song et al., "Towards large language models as copilots for theorem proving in Lean," arXiv:2404.12534 (2024).

[5] Google DeepMind, "AI achieves silver-medal standard solving International Mathematical Olympiad problems," (2024).

Check	Name	What it verifies
1	bundle_figure_integrity	Bundle figures match a fresh render and are legible at typeset
2	bundle_stage13_claim_consistent	No bundle claims stage13_status='green' while the graph sho
3	bundle_reviewer_stage_ordering	No bundle records a Stage-13 verdict before Stages 9 and 10
4	bundle_manuscript_length	Every bundle's compiled manuscript is within its declared len
5	bundle_metadata_matches_graph	bundle_metadata.json finding counts equal the live graph's
6	readiness_verdicts_agree	The heatmap and the submission gate return the same per-bu
7	readiness_submission_gate	Every paper has all P1 readiness gates passed (Phase 5v Wav
8	bundle_consistency	Cross-bundle clusters' member sentences agree on numerical c
9	bundle_registry_consistency	Publication-bundle roster has ONE source of truth (scripts/b
10	bundle_apex_resolves	Every apex theorem a bundle declares names a live Lean theo
11	parameter_provenance	Every experimental parameter has verified provenance
12	citation_primary_sources_present	Every external bibitem cited in papers has a primary-source c
13	provenance_doi_in_registry	PARAMETER_PROVENANCE source DOIs resolve to CITA
14	bibitem_title_primary_source	Registry titles match primary-source cache PDF page-1 titles
15	counts_fresh	counts.json / counts.tex are up-to-date vs. sources
16	tables_fresh	Paper tables (tables/*.tex) are up-to-date vs. pipeline source
17	claim_clusters_fresh	papers/claim_clusters.json is up-to-date vs. v2 claims_review.
18	notebook_stored_outputs_current	Bundle companion notebooks' STORED outputs equal what
19	bundle_source_freshness	Bundle source-paper mtime ≤ bundle last_lift; flag stale bund
20	inventory_index_autogen_fresh	Advisory: SK_EFT_Hawking_Inventory_Index.md autogen blo
21	architecture_inventory_fresh	docs/architecture/SURFACE_INVENTORY.md matches a fr
22	graph_integrity	Knowledge graph integrity — orphans, conflicts, broken chain
23	atlas_integrity	Derived proof-atlas (ADR-005) is consistent: no kind conflicts
24	atlas_hypothesis_discipline	INFO: tracked-hypothesis distribution (total / gating vs orph
25	gate_edge_types_are_emitted	Every edge type a readiness gate queries is actually emitted b
26	formula_grounding	Every formulas.py Lean reference resolves to a real, non-place
27	vacuous_statement_audit	No project theorem/lemma has a content-thin (reflexive / tau
28	nogo_substrate_integrity	Every provably-false no-go has a live, kernel-pure, non-vacuou
29	formulas	Python formulas reference valid Lean theorems
30	placeholder_not_cited	Placeholder (True := trivial) theorems are not cited as verifie
31	disclosure_consistency	No paper presents a disclosed definitional/vacuous_proxy theo
32	proxy_body_audit	Structurally-named theorems are not proved by a trivial 'defi
33	tracked_hypothesis_ledger	Every consumed tracked-hypothesis Prop is registered in HYP
34	tracked_hypotheses_fresh	docs/PERMANENT_TRACKED_HYPOTHESES.md is up-to
35	lean_zero_sorry	Pipeline Invariant #4 — no declaration's axiom closure conta
36	native_decide_regression	native_decide decl-closure does not silently grow past its ceili
37	theorems	Aristotle registry entries resolve to real Lean declarations (ra
38	lean_source	Key theorem names found in Lean source files
39	lean_build	Lean project builds cleanly (requires lake)
40	axiom_closure_allowlist	Every SKEFTHawking declaration's transitive axiom closure
41	elaboration_knob_watchlist	Watchlist (advisory): proof-body maxRecDepth / synthInstan
42	lean_docstring_refs_resolve	Lean docstring 'backticked' project names resolve (rename-dr
43	lean_modules_in_build_graph	Every project .lean module is reachable from the root aggrega
44	notebooks	Notebooks import physics from src.core, no re-implementatio
45	viz_consistency	Notebook visualizations use imported physics and consistent
46	notebook_exec	All notebooks execute without errors
47	paper_provenance	Paper figure references resolve and no placeholder bibliograph
48	numerical_literals	Paper .tex files free of inline numerical literals outside \input
49	count_literals	Paper .tex files reference counts via \input{counts.tex} ma
50	bundle_tables_use_pipeline	Bundle drafts source tables from the pipeline (\input{table
51	paper_latex_compiles	Every papers/*/paper_draft.tex compiles under pdflatex — b
52	axiom_count_prose_consistency	Paper prose axiom-count claims agree with docs/counts.json